Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the constraint satisfaction problem in problem formulation**	
Experiment No: 14	**Date: 20/4/2026**	**Enrolment No: 92301733059**

Aim: To understand the constraint satisfaction problem in problem formulation

IDE: Google Colab

Theory:

The Constraint Satisfaction Problem (CSP) is a fundamental concept in computer science and artificial intelligence, playing a crucial role in solving a wide range of real-world problems. At its core, CSP involves finding a solution to a problem where the solution must satisfy a set of constraints that define the problem's requirements and limitations. These constraints restrict the possible values that variables can take, leading to a search for a combination of values that satisfies all constraints simultaneously.

CSPs can be formulated in various domains, including scheduling, planning, resource allocation, configuration, and design. Examples of CSP applications include job scheduling, timetabling, Sudoku puzzles, map coloring, and the traveling salesman problem. In each case, the problem consists of a set of variables, each with a domain of possible values, and a set of constraints that specify allowable combinations of variable values.

The basic components of a CSP include:

1. **Variables**: These represent the entities whose values need to be determined. Each variable has a domain, which is a set of possible values it can take.
2. **Domains**: A domain is the set of values that a variable can take. Domains can be finite or infinite, discrete or continuous, depending on the problem domain.
3. **Constraints**: Constraints define the relationships among variables and restrict the combinations of values they can take. Constraints can be unary (applying to a single variable), binary (between two variables), or higher-order (involving three or more variables).

The goal in solving a CSP is to find an assignment of values to variables such that all constraints are satisfied. This involves searching through the space of possible assignments to find a solution or determine that no solution exists. The search process typically involves backtracking, constraint propagation, and variable ordering heuristics to efficiently explore the search space.

One common approach to solving CSPs is backtracking search, which systematically explores the space of possible assignments by making choices and backtracking when a dead-end is reached. Backtracking search is guided by variable and value ordering heuristics to prioritize the search for promising solutions and avoid exploring unpromising regions of the search space.

Another technique used in CSP solving is constraint propagation, which involves enforcing constraints locally to reduce the domain of variables and prune the search space. Constraint propagation techniques include arc consistency, domain reduction, and constraint propagation algorithms such as AC-3 and AC-4.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the constraint satisfaction problem in problem formulation**	
Experiment No: 14	**Date: 20/4/2026**	**Enrolment No: 92301733059**

In addition to backtracking search and constraint propagation, other methods for solving CSPs include local search algorithms, genetic algorithms, and constraint satisfaction neural networks. Each approach has its strengths and weaknesses, making them suitable for different types of CSPs and problem domains.

Despite the challenges involved in solving CSPs, they offer a powerful framework for modeling and solving complex combinatorial optimization problems. By formulating real-world problems as CSPs and applying appropriate solving techniques, researchers and practitioners can tackle a wide range of practical problems efficiently and effectively.

Methodology:

1. Mention the constraints for sudoku solution
2. Solve the sudoku using the CSP approach

Program (Code):

```
# Function to check if it is safe to place num at mat[row][col]
def isSafe(mat, row, col, num):

    # Check if num exists in the row
    for x in range(9):
        if mat[row][x] == num:
            return False

    # Check if num exists in the col
    for x in range(9):
        if mat[x][col] == num:
            return False

    # Check if num exists in the 3x3 sub-matrix
    startRow = row - (row % 3)
    startCol = col - (col % 3)

    for i in range(3):
        for j in range(3):
            if mat[i + startRow][j + startCol] == num:
                return False

    return True

# Function to solve the Sudoku problem
def solveSudokuRec(mat, row, col):
    # base case: Reached nth column of the last row
    if row == 8 and col == 9:
        return True
```

Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the constraint satisfaction problem in problem formulation**	
Experiment No: 14	**Date: 20/4/2026**	**Enrolment No: 92301733059**

```
    # If last column of the row go to the next row
    if col == 9:
        row += 1
        col = 0

    # If cell is already occupied then move forward
    if mat[row][col] != 0:
        return solveSudokuRec(mat, row, col + 1)

    for num in range(1, 10):

        # If it is safe to place num at current position
        if isSafe(mat, row, col, num):
            mat[row][col] = num
            if solveSudokuRec(mat, row, col + 1):
                return True
            mat[row][col] = 0
    return False

def solveSudoku(mat):
    solveSudokuRec(mat, 0, 0)

if __name__ == "__main__":
    mat = [
        [3, 0, 6, 5, 0, 8, 4, 0, 0],
        [5, 2, 0, 0, 0, 0, 0, 0, 0],
        [0, 8, 7, 0, 0, 0, 0, 3, 1],
        [0, 0, 3, 0, 1, 0, 0, 8, 0],
        [9, 0, 0, 8, 6, 3, 0, 0, 5],
        [0, 5, 0, 0, 9, 0, 6, 0, 0],
        [1, 3, 0, 0, 0, 0, 2, 5, 0],
        [0, 0, 0, 0, 0, 0, 0, 7, 4],
        [0, 0, 5, 2, 0, 6, 3, 0, 0]
    ]

    solveSudoku(mat)

    for row in mat:
        print(" ".join(map(str, row)))
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the constraint satisfaction problem in problem formulation**	
Experiment No: 14	**Date: 20/4/2026**	**Enrolment No: 92301733059**

Results:

```
3 1 6 5 7 8 4 9 2
5 2 9 1 3 4 7 6 8
4 8 7 6 2 9 5 3 1
2 6 3 4 1 5 9 8 7
9 7 4 8 6 3 1 2 5
8 5 1 7 9 2 6 4 3
1 3 8 9 4 7 2 5 6
6 9 2 3 5 1 8 7 4
7 4 5 2 8 6 3 1 9
```

Observation

In this lab, we successfully modeled and solved a Sudoku puzzle as a Constraint Satisfaction Problem (CSP) using a backtracking search algorithm. The following observations were made:

- **Variables and Domains:** The Sudoku grid consists of 81 variables (cells). The domain for each empty variable is the set of integers {1, 2, 3, 4, 5, 6, 7, 8, 9}.
- **Constraint Enforcement:** The `isSafe` function effectively implemented the three core CSP constraints:
    1. Row Constraint: Each number must be unique in its row.
    2. **Column Constraint:** Each number must be unique in its column.
    3. **Sub-grid Constraint:** Each number must be unique within its 3×3 local matrix.
- **Backtracking Mechanism:** The `solveSudokuRec` function demonstrated a recursive search. When a number was placed that led to a violation of constraints later in the grid, the algorithm correctly "backtracked" by resetting the cell to 0 and trying the next available number in the domain.
- **Maze Analogy:** While the Sudoku code solves a grid, the logic is analogous to solving the provided **maze image**. In a maze, the variables are the "junctions/steps," and the constraints are the "walls" that restrict movement.

Result Analysis

The execution of the program yielded a valid completed Sudoku grid, which leads to the following analysis:

- **Efficiency of CSP:** CSP formulation significantly reduces the search space compared to a pure brute-force approach. By checking `isSafe` before recursing, we prune branches of the search tree that are guaranteed to fail.
- **Completeness:** The backtracking algorithm is **complete**, meaning that if a solution exists for the given initial state (the provided `mat`), the algorithm is guaranteed to find it.

Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the constraint satisfaction problem in problem formulation**	
Experiment No: 14	**Date: 20/4/2026**	**Enrolment No: 92301733059**

- **Search Complexity:** For the provided puzzle, the algorithm reached the base case (`row == 8 and col == 9`) successfully. However, the time complexity of this approach is $O(9^n)$, where n is the number of empty cells. While efficient for 9×9 Sudoku, larger CSPs might require more advanced techniques like **Arc Consistency (AC-3)** or **Minimum Remaining Values (MRV)** heuristics to improve performance.
- **Conclusion:** The lab demonstrates that formulating a problem with clear variables, domains, and constraints allows for a structured search. This approach is highly versatile and can be adapted to solve various logic puzzles and real-world scheduling problems.

Post Lab Exercise:

Write the code and output for N-Queens problem. Take the value of N from the user.

```
def is_safe(board, row, col, n):
    # Check this row on left side
    for i in range(col):
        if board[row][i] == 1:
            return False

    # Check upper diagonal on left side
    for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
        if board[i][j] == 1:
            return False

    # Check lower diagonal on left side
    for i, j in zip(range(row, n, 1), range(col, -1, -1)):
        if board[i][j] == 1:
            return False

    return True

def solve_n_queens_util(board, col, n):
    # Base case: If all queens are placed
    if col >= n:
        return True

    # Consider this column and try placing this queen in all rows one by one
    for i in range(n):
        if is_safe(board, i, col, n):
            board[i][col] = 1 # Place queen

            # Recur to place rest of the queens
            if solve_n_queens_util(board, col + 1, n):
                return True
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the constraint satisfaction problem in problem formulation**	
Experiment No: 14	**Date: 20/4/2026**	**Enrolment No: 92301733059**

```
            # If placing queen in board[i][col] doesn't lead to a solution,
            # then backtrack: remove queen from board[i][col]
            board[i][col] = 0
    return False
def solve_n_queens():
    try:
        n = int(input("Enter the value of N: "))
        if n <= 0:
            print("Please enter a positive integer.")
            return
    except ValueError:
        print("Invalid input. Please enter a number.")
        return

    # Initialize board with 0s
    board = [[0 for _ in range(n)] for _ in range(n)]

    if not solve_n_queens_util(board, 0, n):
        print(f"Solution does not exist for N = {n}")
        return
    print(f"\nSolution for {n}-Queens:")
    for row in board:
        print(" ".join(["Q" if x == 1 else "." for x in row]))

if __name__ == "__main__":
    solve_n_queens()
```

Outputs

```
•••  Enter the value of N: 4

     Solution for 4-Queens:
     . . Q .
     Q . . .
     . . . Q
     . Q . .
```